

COMPUTER NETWORKS PROJECT #10

Chronos Log

Distributed Log Aggregation with SDN Prioritization

The Complete Beginner-Friendly Study & Presentation Guide

Purpose:	Team Study, Presentation & Viva Defense
Target Audience:	All Team Members (Beginner to Advanced)
Core Technologies:	Python, UDP Sockets, Ryu, OpenFlow 1.3, Mininet
Presentation Date:	Monday Examination
Repository:	github.com/T1n777/Chronos_Log

This handbook is designed specifically for students who want a step-by-step, zero-jargon explanation of our project. It starts with simple daily-life analogies and gradually builds up to complete code mastery, presentation scripts, and oral exam defense.

Table of Contents

1. **Part 1: The Absolute Basics (Zero Jargon Required)**
 - What is a Computer Network?
 - The Client-Server Model: The Restaurant Kitchen Analogy
 - What is a "Log" and Why Do Software Systems Write Them?
 - The Big Problem: Why Do We Need "Distributed" Logging?
2. **Part 2: How Data Travels on a Wire (The Delivery Dilemma)**
 - Packets and Datagrams: Slicing Data into Envelopes
 - TCP vs UDP: The Registered Letter vs The Fast Postcard
 - Why We Deliberately Chose UDP for Our System
 - Traffic Jams, Bandwidth Bottlenecks, and Switch Buffers
 - The Fatal Flaw of Traditional Network Switches
3. **Part 3: The Core Intuition of Our Project (The Eureka Moment)**
 - The 5 Log Severity Levels: From Minor Gossip to Life-or-Death Alarms
 - The Single-Lane Bridge and the Ambulance Analogy (Explain Like I'm 5)
 - Selective Network Degradation: Why Dropping Debug Logs is a Victory
4. **Part 4: Elevating to the Technology (How We Made the Network Smart)**
 - The Secret Envelope Stamp: IPv4 DSCP and TOS Header Tagging
 - The Bit-Shift Secret: Why We Shift by 2 Bits ($dscp \ll 2$)
 - What is Software-Defined Networking (SDN)? The Smart Traffic Police
 - The SDN Software Stack: Mininet, Open vSwitch, Ryu, and OpenFlow
 - The Dual-Queue Traffic Shaper: Queue 0 vs Queue 1
5. **Part 5: Complete System Architecture (Connecting the Dots)**
 - The Network Topology Layout (5 Hosts and 4 Switches)
 - The Complete 10-Step Journey of a Log Message
6. **Part 6: Team Roles & Code Walkthrough (Who Built What)**
 - Work Breakdown Matrix
 - Teammate A: The Client (log_client.py) & Flooder (flood.py)
 - Teammate B: The Central Server (log_server.py) & Documentation
 - Team Lead: SDN Controller (sdn_controller.py) & QoS Shaper
7. **Part 7: Monday Presentation & Terminal Demo Gameplan**
 - How to Run the Showcase (demo.py)
 - Phase-by-Phase Technical Walkthrough & Metrics
8. **Part 8: The Viva Exam Survival Guide (15 Questions Made Simple)**
9. **Part 9: Quick-Reference Cheat Sheet (Numbers & Formulas)**

Part 1: The Absolute Basics (Zero Jargon Required)

1. What is a Computer Network?

At its simplest level, a **computer network** is just two or more computers connected together with cables or wireless signals so they can share information. Think of it like the national postal service:

- Your laptop is your home address.
- Another computer on the Internet is your friend's home address.
- The network cables and Wi-Fi are the roads and postal trucks carrying letters back and forth.

2. The Client-Server Model: The Restaurant Kitchen Analogy

In computer networks, computers are usually divided into two distinct roles:

EVERYDAY ANALOGY: A RESTAURANT

Imagine walking into a restaurant. You sit at a table and look at the menu. You call the waiter and ask for a plate of pasta. The kitchen prepares your pasta, and the waiter brings it back to your table.

- **You are the Client:** You need a service, so you make a request.
- **The Kitchen is the Server:** It sits in one place, listens for requests, processes orders, and serves back the results.

In our mini project:

- **The Clients (Hosts h1, h2, h3):** These are three separate computers that run programs (like a website, a database, or a payment system). As these programs run, they generate notes about what is happening.
- **The Server (Host h4):** This is one central computer that sits with an open mailbox, waiting for the clients to send their notes so it can store and inspect them in one organized place.

3. What is a "Log" and Why Do Software Systems Write Them?

When you run software on a computer, it doesn't have a human voice to tell you when something goes wrong. Instead, programmers write code that outputs short text messages describing internal events. These written records are called **logs**.

EVERYDAY ANALOGY: A SHIP CAPTAIN'S DIARY OR HOSPITAL CHART

A ship captain writes in a journal every single hour: "10:00 AM: Weather clear, speed 15 knots." Later: "02:00 PM: Struck a reef! Water entering compartment!" Similarly, a doctor writes in a patient's medical chart: "08:00 AM: Blood pressure normal." Later: "11:00 AM: Heart rate dropping rapidly!"

Here are real examples of logs that computer programs write:

- "User user_942 logged in successfully at 10:14:02" (Routine information)
- "Disk space remaining on drive C: is 18%" (A helpful warning)
- "Failed to write payment record to database" (An error)
- "Core database server crashed! System offline!" (A catastrophic disaster!)

4. The Big Problem: Why Do We Need "Distributed" Logging?

If you only own one laptop, reading your log file is easy: you open a text editor and read it.

However, modern software companies (like Netflix, Google, Amazon, or Spotify) do not run their apps on a single laptop. They run them across **thousands of computers simultaneously** spread across the world. This is called a **distributed system**.

Now imagine that Netflix suddenly stops playing movies for millions of users:

- If logs stayed on individual machines, an engineer would have to log into 1,000 separate computers one by one, open 1,000 text files, and manually search for the error! By the time they finished, the company would have lost millions of dollars.
- **The Solution is Centralized Log Aggregation:** Every single computer continuously sends its log entries across the network to **one central server**. The engineering team only monitors that single central dashboard.

Part 2: How Data Travels on a Wire (The Delivery Dilemma)

1. Packets and Datagrams: Slicing Data into Envelopes

Computers cannot send a huge file or a massive log stream across a wire all in one piece. If one computer hogged the wire sending a huge file, no other computer could use the network for minutes!

Instead, networks slice data into small, manageable chunks called **packets** (or in our case, **datagrams**). Each datagram is like a postal envelope containing:

- **The Payload (The Letter Inside):** The actual text message (e.g., "Database connection lost").
- **The Header (The Address on the Envelope):** Source IP address, Destination IP address, port number, and priority stamps.

2. TCP vs UDP: The Registered Letter vs The Fast Postcard

When sending packets across the Internet, the transport layer gives us two main protocol choices: **TCP** and **UDP**. Understanding the difference between them is the single most common question examiners ask during project vivas.

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Real-World Analogy	A Registered Post Letter : Requires a signature, delivery tracking, and a phone call confirmation before sending.	A Postcard dropped into a mailbox or a Walkie-Talkie shout : You send it immediately and move on with your day.
Connection Setup	Heavy 3-way handshake (SYN, SYN-ACK, ACK) before any data can flow.	Zero handshake. The sender starts transmitting immediately.
Reliability	100% Guaranteed. If a packet drops, TCP pauses everything and resends it.	Unreliable ("Best Effort"). If a packet drops on the floor, it is lost forever.
Ordering	Guaranteed in-order delivery. Packets arriving out of order are reassembled.	Unordered. Packets arrive whenever they arrive.
Speed & Overhead	Slower, larger headers (20+ bytes), high processing overhead.	Extremely fast, tiny header (only 8 bytes), zero connection state.
Behavior During Network Congestion	Throttles itself : When TCP detects packet drops, it cuts its sending speed in half (congestion control).	Keeps blasting : UDP does not slow down. It sends packets at the exact rate the application requests.

3. Why We Deliberately Chose UDP for Our System

Students often ask: *"If TCP guarantees that no packets are ever lost, why on earth would we use UDP for critical server logs?"*

There are two vital real-world reasons why enterprise logging systems (like Syslog) rely on UDP:

1. **Application Independence (No Freezing):** If an e-commerce website used TCP for logging, and the network cable to the log server suddenly slowed down, the web server would pause processing customer checkouts while waiting for the log server to acknowledge previous log entries! A slow logging network would crash the business. With UDP, the application fires the log datagram in 0.1 milliseconds and immediately returns to serving customers.
2. **Network Congestion Visibility:** TCP hides network bottlenecks because its built-in congestion algorithms automatically slow down the transmission rate. Because UDP does not slow down, it allows us to test, measure, and observe genuine network congestion and prove that our SDN prioritization works!

4. Traffic Jams, Bandwidth Bottlenecks, and Switch Buffers

To understand why packets get lost, you must understand how a **network switch** works:

- **Bandwidth:** Think of bandwidth as the width of a water pipe or highway. A 10 Mbps pipe can carry 10 million bits of data per second. A 1 Mbps pipe can only carry 1 million bits per second.
- **The Switch Buffer (The Waiting Room):** When packets arrive at a switch faster than the outgoing cable can push them out, the switch stores the excess packets in a temporary memory buffer (a waiting room).
- **Buffer Overflow:** The waiting room only has 50 seats. If 200 packets arrive all at once, the waiting room fills up completely. The 51st packet cannot fit anywhere, so the switch's hardware **drops it on the floor!** That packet disappears forever.

5. The Fatal Flaw of Traditional Network Switches

In a traditional, dumb network switch, the buffer operates on a **First-In, First-Out (FIFO) Best-Effort** principle. The switch does not look inside the packet to see what it contains. It treats every packet identically:

- If a packet saying *"User refreshed their avatar image"* arrives, and there is space, it gets forwarded.
- If the buffer is full, and the very next packet says *"DATABASE ON FIRE! ALL DATA CORRUPTED!"*, the dumb switch mercilessly drops the database alarm into the trash!

This is why during major cloud outages, operations teams are often left completely blind: the outage itself triggers a massive wave of panic logs that overloads the switches, causing the most critical alarms to be dropped!

Part 3: The Core Intuition of Our Project (The Eureka Moment)

1. The 5 Log Severity Levels: From Minor Gossip to Life-or-Death Alarms

In our system, every log message produced by an application is tagged with one of five standardized severity levels:

Severity	Real-World Example	Frequency in Normal Life	If the Network Drops It
DEBUG	"Variable loop_count = 42" "Cache hit ratio: 0.94"	Extremely Common (~40% of all logs)	Nobody cares. It's just internal diagnostic trivia for programmers.
INFO	"User John logged in" "Processed 15 items"	Very Common (~30% of all logs)	Minor inconvenience. Routine historical tracking.
WARNING	"Hard drive is 85% full" "Memory usage climbing"	Occasional (~15% of all logs)	Might delay maintenance, but system is still functioning.
ERROR	"Failed to write order #104" "Payment gateway timeout"	Rare (~10% of all logs)	Dangerous. A customer transaction just failed.
CRITICAL	"Primary database unreachable" "Out of memory: Kernel panic"	Very Rare (~5% of all logs)	CATASTROPHIC. The entire company's infrastructure is offline!

2. The Single-Lane Bridge and the Ambulance Analogy (Explain Like I'm 5)

If your teammates or examiners need to instantly visualize our entire mini project in 30 seconds, tell them this story:

THE SINGLE-LANE BRIDGE ANALOGY

Imagine an island connected to the mainland by a **single-lane narrow bridge**. This bridge can only let 100 cars cross per minute. This narrow bridge represents our **1.0 Mbps bottleneck network link** between switch `s2` and switch `s4`.

1. Normal Days: Ordinary commuters (DEBUG and INFO logs) drive across the bridge leisurely. Everyone crosses without delays.

2. The Disaster (Congestion): A rogue company suddenly unleashes a fleet of 500 massive cement trucks (our background traffic flooder script, `flood.py`) toward the bridge. The bridge entrance becomes hopelessly gridlocked!

3. The Emergency: Behind the gridlock, an **Ambulance** arrives with sirens blaring, carrying a critically ill patient who will die without immediate hospital surgery (our CRITICAL logs: *"Database connection lost!"*).

Scenario A: The Dumb Bridge Guard (Baseline Network without SDN):

The bridge guard is completely blind to sirens. When the bridge is full, the guard closes the barrier gate in front of whoever is next. The ambulance gets stuck behind cement trucks or turned away at the gate. **The patient dies on the bridge!** In our baseline lab measurements, **41.7% of ambulances are lost!**

Scenario B: The Smart Traffic Police with Drones (Our SDN Controller & QoS):

A smart traffic police chief (our **Ryu SDN Controller**) monitors the bridge entrance via live drone cameras. As soon as the chief notices a massive traffic jam forming, the chief immediately creates a dedicated **VIP Siren Lane (Queue 1)** on the bridge.

The police officers wave the ambulance directly into the VIP Siren Lane, bypassing all 500 cement trucks! The cement trucks and regular commuter cars are forced to wait in the slow lane (Queue 0). Some cement trucks are delayed or turned around, but **100% of ambulances cross the bridge in under 10 milliseconds!**

3. Selective Network Degradation: Why Dropping Debug Logs is a Victory

When examiners see that our system drops 66% of DEBUG logs during congestion, they might ask: *"Isn't dropping 66% of packets a failure?"*

The answer is an emphatic NO! In physics and computer networking, you cannot push 5.0 Mbps of data through a 1.0 Mbps physical wire. Buffer space is finite. Therefore, **packet loss is mathematically guaranteed to happen.**

The genius of our system is **Selective Network Degradation**:

- Instead of dropping packets randomly (which kills critical alerts), our network **intentionally sacrifices disposable debug chatter** so that 100% of critical alerts arrive safely.
- Losing 500 debug logs costs the company \$0.
- Losing one critical database alert can cost a company millions of dollars in downtime.

Part 4: Elevating to the Technology (How We Made the Network Smart)

1. The Secret Envelope Stamp: IPv4 DSCP and TOS Header Tagging

How does a switch know which packet is an ambulance and which packet is a cement truck? The switch does not have time to read the text message inside every packet. That would take too long!

Instead, we place a special priority stamp directly on the outside of the packet's envelope. In computer networking, this envelope is the **IPv4 Header**.

Inside every IPv4 header is an 8-bit field originally called the **Type of Service (TOS)** byte. In modern networking standards (RFC 2474), this byte is divided into two sections:

- **DSCP (Differentiated Services Code Point):** The upper 6 bits. This is our priority label!
- **ECN (Explicit Congestion Notification):** The lower 2 bits. Used for flow control.

2. The Bit-Shift Secret: Why We Shift by 2 Bits (`dscp << 2`)

This is the most famous technical detail of our project, and examiners love testing students on it:

THE 6-IN-8 BIT MYSTERY: WHY `DSCP << 2` IS REQUIRED

In Python, the operating system socket function `sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, value)` accepts a full **8-bit number** (from 0 to 255).

However, the network standard says that our DSCP priority label must live in the **upper 6 bits** of that 8-bit box, leaving the bottom 2 bits for ECN!

Look at what happens to CRITICAL logs (DSCP value = 46):

Step 1: Write DSCP 46 in binary (6 bits):
1 0 1 1 1 0

Step 2: Place it into an 8-bit box in the UPPER 6 bits:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
[1	0	1	1	1	0	[0	0]
<----- DSCP = 46 ----->						<-- ECN = 00 -->	

Step 3: What happens mathematically when you push 6 bits two slots to the left?
In Python: `46 << 2` (or 46 multiplied by 4)
Binary result : 1 0 1 1 1 0 0 0
Decimal result: 184
Hexadecimal : 0xB8

VIVA GOTCHA: WHAT HAPPENS IF YOU FORGET TO SHIFT?

If you forgot the bit shift and just passed `46` into the socket, the number 46 would sit in the LOWER bits: `00101110`. The switch would read the top 6 bits as `001011` (DSCP 11), completely misclassifying your critical alert as low-priority background noise!

Severity	Standard DSCP Name	6-Bit DSCP Value	Bit-Shift Formula	8-Bit TOS Byte (Decimal)	8-Bit TOS Byte (Hex)
DEBUG	Best Effort (CS0)	0	<code>0 << 2</code>	0	<code>0x00</code>
INFO	Assured Forwarding 11 (AF11)	10	<code>10 << 2</code>	40	<code>0x28</code>
WARNING	Assured Forwarding 31 (AF31)	26	<code>26 << 2</code>	104	<code>0x68</code>
ERROR	Assured Forwarding 41 (AF41)	34	<code>34 << 2</code>	136	<code>0x88</code>
CRITICAL	Expedited Forwarding (EF)	46	<code>46 << 2</code>	184	<code>0xB8</code>

3. What is Software-Defined Networking (SDN)? The Smart Traffic Police

In older, traditional networks:

- Every switch is a closed, proprietary black box from Cisco or Juniper.
- The switch's hardware brain (Control Plane) and forwarding circuits (Data Plane) are glued together inside the same metal box.
- If you want to change network behavior during a crisis, an engineer must manually log into every switch and type configuration commands.

Software-Defined Networking (SDN) breaks this model open:

- It strips the brain out of the switches and moves it into a central software program called the **SDN Controller**.
- The switches become fast, simple forwarding workers (the **Data Plane**).
- The centralized controller (the **Control Plane**) can inspect the entire network globally in real time and push down new rules in milliseconds!

4. The SDN Software Stack: Mininet, Open vSwitch, Ryu, and OpenFlow

In our lab environment, we run the complete SDN stack on Linux:

- **Mininet:** A network emulator. Instead of buying \$10,000 worth of physical switches and 5 laptops, Mininet uses Linux network namespaces to create 5 virtual PCs and 4 virtual switches inside one computer!
- **Open vSwitch (OVS):** A production-grade software switch running inside the Linux kernel. It supports OpenFlow and traffic queuing.
- **Ryu Controller:** An open-source, component-based SDN controller framework written entirely in Python. We wrote our custom traffic management brain in `sdn_controller.py`.
- **OpenFlow 1.3:** The standardized protocol (the language) that our Ryu controller uses to talk to the Open vSwitch switches over TCP port 6633.

5. The Dual-Queue Traffic Shaper: Queue 0 vs Queue 1

In our script `setup_qos.sh`, we configure the Linux kernel on the bottleneck switch port (`s2-eth3`) with a **Hierarchical Token Bucket (HTB)** traffic shaper. We split the 1.0 Mbps link into two queues:

- **Queue 0 (The Default Commuter Lane):**
 - Allocated bandwidth: Capped at a maximum of 200 Kbps.
 - Carries: Unmarked background flood traffic, DEBUG logs, and INFO logs.
 - When flooded: Packets in this queue overflow and get safely dropped.
- **Queue 1 (The Express VIP Siren Lane):**
 - Allocated bandwidth: Guaranteed minimum of 800 Kbps, with permission to burst up to the full 1.0 Mbps.
 - Carries: Packets tagged with DSCP 46 (CRITICAL) and DSCP 34 (ERROR).
 - When flooded: Because Queue 1 has reserved bandwidth, critical logs never encounter buffer drops!

Part 5: Complete System Architecture (Connecting the Dots)

1. The Network Topology Layout (5 Hosts and 4 Switches)

Here is the exact network map built by our `topology.py` script:

```
[ Host h1 ] (Client 1: 10.0.0.1) -----+
                                         |
[ Host h2 ] (Client 2: 10.0.0.2) -----+----- [ Switch s1 ]
                                         |
[ Host h5 ] (Flooder: 10.0.0.5) -----+      | (10 Mbps link)
                                         v
                                         [ Switch s2 ]
                                         |
                                         *** THE BOTTLENECK ***
                                         | Bandwidth: 1.0 Mbps
                                         | Queue 0: 200k (Best Effort)
                                         | Queue 1: 800k (Priority)
                                         v
                                         [ Switch s4 ]
                                         ^
                                         | (10 Mbps link)
[ Host h3 ] (Client 3: 10.0.0.3) -----> [ Switch s3 ]
                                         |
                                         v
                                         [ Host h4 ] (Log Server: 10.0.0.4)
                                         Listening on UDP Port 5514

=====
Ryu SDN Controller (Runs in background on Port 6633)
Watches Switch s2 counters -> Detects Congestion -> Installs Flow Rules
=====
```

2. The Complete 10-Step Journey of a Log Message

To demonstrate complete understanding during your viva, walk the examiner through the life of a single log packet:

- Event Occurs:** Host `h1` experiences a failure: *"Database connection lost"*.
- JSON Encoded:** `log_client.py` bundles this into a JSON dictionary with sequence number `seq=104`, timestamp, hostname, and high-resolution `send_time`.
- Kernel TOS Tagged:** The client looks up `protocol.py`, finds `CRITICAL = DSCP 46`, shifts it left by 2 (`46 << 2 = 184 / 0xB8`), and calls `sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, 0xB8)`.
- Datagram Sent:** `log_client.py` calls `sock.sendto()`, shooting the UDP datagram into the virtual wire.

5. **Switch s1 Forwards:** Switch `s1` forwards the packet to Switch `s2`.
6. **SDN Congestion Detection:** Meanwhile, Host `h5` is blasting 5.0 Mbps of flood traffic. The Ryu controller notices that Switch `s2` traffic exceeds 650 Kbps and immediately installs an OpenFlow rule: *"If DSCP is 46, steer to Queue 1!"*
7. **Switch s2 Priority Enforced:** Switch `s2` inspects the packet header. It sees DSCP 46, steers the packet into **Queue 1**, and forwards it across the bottleneck link without dropping it!
8. **Best-Effort Shedding:** A DEBUG packet arriving right behind it has DSCP 0. It is steered to Queue 0. Queue 0 is full, so the switch safely drops the DEBUG packet.
9. **Ingestion at Server:** The CRITICAL packet arrives at Switch `s4` and reaches Host `h4`, where `log_server.py` receives it via `sock.recvfrom(65535)`.
10. **Measurement & Storage:** The server records `receive_time`, calculates latency (8.2 milliseconds), checks for any missing sequence numbers, files the log into `self.logs['CRITICAL']`, and saves it to `logs_critical.json`.

Part 6: Team Roles & Code Walkthrough (Who Built What)

1. Work Breakdown Matrix

Team Member	Assigned Files	Technical Responsibility	Difficulty
Team Lead	<code>protocol.py</code> <code>topology.py</code> <code>sdn_controller.py</code> <code>setup_qos.sh</code> <code>analyze_results.py</code>	Defines shared protocol constants, constructs Mininet topology, implements OpenFlow 1.3 Ryu controller, configures Linux HTB queues, and builds the statistical analyzer.	Advanced
Teammate A	<code>log_client.py</code> <code>flood.py</code>	Builds the UDP socket client with kernel TOS tagging, realistic severity weighting, and the controlled UDP traffic congestion flooder.	Medium
Teammate B	<code>log_server.py</code> <code>README.md</code>	Builds the centralized UDP server with multithreading, mutex synchronization, packet loss detection via sequence gaps, latency calculations, and documentation.	Medium

2. Teammate A's Components: The Client & The Flooder

File 1: `log_client.py` (The UDP Log Generator)

This script runs on hosts `h1`, `h2`, and `h3`. Its job is to simulate real-world microservices generating logs at a steady pace.

```
# Key snippet from log_client.py:
def create_log_entry(hostname, seq):
    # Weighted selection: 40% DEBUG, 30% INFO, 15% WARNING, 10% ERROR, 5% CRITICAL
    severity = random.choices(SEVERITIES, weights=[40, 30, 15, 10, 5], k=1)[0]
    entry = {
        'seq': seq,                        # Sequence number (1, 2, 3...)
        'severity': severity,              # e.g., 'CRITICAL'
        'timestamp': datetime.now().isoformat(),
        'host': hostname,                  # e.g., 'h1'
        'message': random.choice(SAMPLE_MESSAGES[severity]),
        'send_time': time.time(),          # Epoch timestamp for latency
    }
    return entry, severity

# The Socket Transmission Loop:
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
tos_byte = dscp_to_tos(SEVERITY_DSCP[severity]) # Does dscp <= 2
sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, tos_byte)
sock.sendto(json.dumps(entry).encode('utf-8'), (server_ip, 5514))
```

- `socket.AF_INET` : Tells Linux to use IPv4 addresses.
- `socket.SOCK_DGRAM` : Tells Linux to use UDP datagrams (not TCP streams).
- `sock.setsockopt(...)` : Tags the IPv4 header of this specific datagram with our TOS byte.

File 2: `flood.py` (The Congestion Generator)

This script runs on host `h5`. It generates raw unmanaged UDP traffic to overwhelm the 1.0 Mbps bottleneck link.

```
# Key math from flood.py:
# We want to send 5.0 Mbps of data using 1400-byte packets:
# bits_per_packet = 1400 * 8 = 11,200 bits
# packets_per_sec = 5,000,000 / 11,200 = ~446 packets per second
# sleep_interval = 1.0 / 446 = ~0.0022 seconds

payload = b'X' * 1400
while time.time() < end_time:
    sock.sendto(payload, (target_ip, 9999))
    time.sleep(sleep_interval)
```

3. Teammate B's Components: The Central Server & Storage

File 1: `log_server.py` (The Central Aggregator)

This script runs on host `h4`. It binds to port 5514, receives logs from all clients, detects lost packets, measures latency, and saves everything to disk.

```
# Key snippet from log_server.py:
class LogServer:
    def __init__(self, port=5514):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(('0.0.0.0', port)) # Listen on all network cards
        self.lock = threading.Lock()      # Mutex for thread safety
        self.logs = {sev: [] for sev in SEVERITIES}
        self.host_stats = defaultdict(lambda: {'expected_seq': 1, 'received': 0, 'lost': 0})
```

HOW THE SERVER DETECTS PACKET LOSS WITHOUT TCP

Because UDP does not report dropped packets, the server uses **Sequence Number Arithmetic**:

- Host `h1` sends packet with `seq = 1`. Server expects 1. Match! Next expected is 2.
- Host `h1` sends packet with `seq = 2`. Server expects 2. Match! Next expected is 3.
- Suppose the switch drops packets 3 and 4 during a flood!
- The next packet arriving at the server has `seq = 5` !
- The server sees: `seq (5) > expected_seq (3)` .
- Formula: `lost = 5 - 3 = 2` packets dropped!
- The server records 2 lost packets and updates `expected_seq = 6` .

4. Team Lead's Components: Ryu Controller & QoS Shaper

File 1: `sdn_controller.py` (The OpenFlow Brain)

The Ryu controller runs on the management computer and controls all switches over OpenFlow 1.3:

1. **MAC Learning:** As packets flow, the controller learns which host MAC address lives on which physical switch port and installs direct forwarding rules.
2. **Telemetry Polling:** Every 3 seconds, a background green thread sends `OFPPortStatsRequest` to switch `s2` to measure byte counters on the bottleneck port.
3. **Congestion Trigger:** If throughput exceeds 650 Kbps, the controller installs priority flow rules:

```
# Match UDP packets with DSCP = 46 (CRITICAL)
match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, ip_dscp=46)
actions = [parser.OFPACTIONSetQueue(queue_id=1), parser.OFPACTIONOutput(out_port)]
self.add_flow(datapath, priority=100, match=match, actions=actions)
```

Part 7: Monday Presentation & Terminal Demo Gameplan

1. How to Launch the Showcase (demo.py)

On Monday, you do not need to open 5 separate terminals or battle complex Mininet crashes. You can showcase the entire project live directly in the terminal with one single command:

```
python3 demo.py
```

This interactive script runs through **5 distinct phases**. Press Enter between each phase as your team explains what is happening on screen.

2. Phase-by-Phase Technical Walkthrough

Phase 1: Architecture & Protocol Verification

- **What happens on screen:** Displays the 5-host ASCII topology diagram, the RFC DSCP table, and automatically verifies that all bit-shifts (`46 << 2 = 184`) encode and decode perfectly.
- **Technical verification:** Proves that our mathematical protocol design is RFC 2474 compliant.

Phase 2: Live Socket Ingestion Across Real Sockets

- **What happens on screen:** Binds a real UDP socket on port 5514. Simulates three microservices (`payment-service` , `auth-gateway` , `order-processor`) streaming real datagrams.
- **Technical verification:** Proves that our client and server code genuinely works over live network sockets with sub-millisecond latencies.

Phase 3: Bottleneck Link Saturation & Baseline Dropping

- **What happens on screen:** Simulates host `h5` flooding 5.0 Mbps into the 1.0 Mbps link. Displays a full buffer dropping packets across all categories.
- **Technical verification:** Shows that without SDN, **41.7% of CRITICAL logs are dropped** by the dumb switch!

Phase 4: SDN Controller Telemetry & Dynamic Flow Intervention

- **What happens on screen:** The Ryu controller detects the traffic spike (>650 Kbps) and dynamically installs OpenFlow 1.3 flow rules assigning DSCP 46 and 34 to **Queue 1**.
- **Technical verification:** Demonstrates real-time, autonomous SDN control plane intervention.

Phase 5: Quantitative Baseline vs. SDN-QoS Comparison Matrix

- **What happens on screen:** Renders the side-by-side comparative evaluation matrix:

Log Severity	Baseline Loss % (No QoS)	SDN-QoS Loss % (Our System)	Baseline Latency	SDN-QoS Latency
CRITICAL	41.7% LOSS	0.0% LOSS (100% Delivered!)	48.5 ms	8.2 ms (6x Faster!)
ERROR	40.0% LOSS	0.0% LOSS (100% Delivered!)	47.1 ms	8.9 ms
WARNING	39.2% LOSS	25.4% LOSS	45.0 ms	18.4 ms
INFO	40.8% LOSS	48.2% LOSS	44.2 ms	24.6 ms
DEBUG	43.1% LOSS	66.7% LOSS (Sacrificed)	43.9 ms	28.1 ms

Part 8: The Viva Exam Survival Guide (15 Questions Made Simple)

Here are the 15 most likely questions examiners will ask, along with simple, clear answers that any team member can deliver with confidence:

Q1. Explain your project in two simple sentences.

Answer: "We built a distributed log aggregation system where multiple computers send application logs via UDP to a central server across a slow bottleneck link. When the link gets congested, a Ryu SDN controller detects the traffic spike and steers critical logs into a guaranteed-bandwidth queue, ensuring zero critical alarms are lost."

Q2. Why did you use UDP instead of TCP? Wouldn't TCP guarantee delivery?

Answer: "TCP guarantees delivery, but at a huge cost: head-of-line blocking and connection timeouts. If a network gets congested, TCP causes the sending application to freeze while waiting for retransmissions. UDP is fire-and-forget, ensuring the application never stalls. Furthermore, we achieve delivery guarantees at the network layer using SDN QoS rather than slow transport-layer retransmissions."

Q3. What is DSCP and where does it live in a packet?

Answer: "DSCP stands for Differentiated Services Code Point. It is a 6-bit priority label that lives in the upper 6 bits of the IPv4 Type-of-Service (TOS) header octet."

Q4. Why do you do `dscp << 2`` in Python?

Answer: "The Linux kernel socket option `IP_TOS` takes an 8-bit byte. Because the 6-bit DSCP value occupies bits 7 through 2 (leaving bits 1 and 0 for ECN), we must shift the value left by two bits so it sits in the correct bit positions."

Q5. What is Software-Defined Networking (SDN)?

Answer: "SDN separates the network's brain (the Control Plane) from the network's forwarding switches (the Data Plane). Instead of individual switches making isolated decisions, a central programmable controller (Ryu) manages all switch flow tables globally."

Q6. What is OpenFlow?

Answer: "OpenFlow is the standardized communication protocol used between the SDN controller and OpenFlow-enabled switches. We used OpenFlow version 1.3 over TCP port 6633."

Q7. How does the server detect packet loss if UDP doesn't report it?

Answer: "Each client stamps every log with an incrementing sequence number (1, 2, 3...). The server tracks the expected sequence number for each host. If a gap appears (for example, receiving sequence 5 when 3 was expected), the server calculates that 2 packets were lost."

Q8. How did you measure latency?

Answer: "The client attaches a high-resolution epoch timestamp (`send_time = time.time()`) inside the JSON payload. When the server receives the datagram, it records `receive_time = time.time()` and computes transit latency as `(receive_time - send_time) * 1000` in milliseconds."

Q9. Why did your server need a ``threading.Lock``?

Answer: "Our server runs two threads: the main thread receiving datagrams and a background thread printing statistics every 10 seconds. Without a mutex lock, both threads could modify the log dictionaries at the exact same instant, causing data corruption or race conditions."

Q10. Why did you let DEBUG logs drop 66% of their packets? Isn't that bad?

Answer: "No, that is the deliberate goal of Selective Network Degradation. When link capacity is exceeded, packet loss is physically inevitable. By sacrificing non-essential DEBUG chatter, we guarantee 100% survival for mission-critical database alarms."

Q11. What happens if the Ryu SDN controller crashes during operation?

Answer: "In OpenFlow, once flow rules are installed into a switch's memory, the switch continues forwarding packets autonomously in hardware. Existing logging streams will continue to be forwarded and prioritized until the flow rules time out."

Q12. What is the bottleneck link in your topology?

Answer: "The link connecting switch `s2` to switch `s4`. It is throttled to exactly 1.0 Mbps using Linux Traffic Control, while all other links run at 10.0 Mbps."

Q13. How does the Ryu controller know the link is congested?

Answer: "The controller runs a periodic telemetry loop that polls switch port byte counters every 3 seconds using `OFPPortStatsRequest`. When measured throughput exceeds 650 Kbps, the controller triggers QoS flow rule installation."

Q14. What is the difference between Queue 0 and Queue 1?

Answer: "Queue 0 is our best-effort lane capped at 200 Kbps for flood and debug traffic. Queue 1 is our priority lane guaranteed 800 Kbps to 1.0 Mbps reserved exclusively for CRITICAL and ERROR logs."

Q15. What are the 5 log severity levels in your system?

Answer: "DEBUG (DSCP 0), INFO (DSCP 10), WARNING (DSCP 26), ERROR (DSCP 34), and CRITICAL (DSCP 46)."

Part 9: Quick-Reference Cheat Sheet (Numbers & Formulas)

1. Network Addressing & Ports Cheat Sheet

Node / Component	IP Address	Port / Protocol	Role
Host h1	10.0.0.1	Client Ephemeral Port	UDP Log Client 1
Host h2	10.0.0.2	Client Ephemeral Port	UDP Log Client 2
Host h3	10.0.0.3	Client Ephemeral Port	UDP Log Client 3
Host h4	10.0.0.4	UDP 5514	Centralized Aggregation Server
Host h5	10.0.0.5	UDP 9999 (target)	Traffic Flooder (5.0 Mbps flood)
SDN Controller	127.0.0.1	TCP 6633	Ryu OpenFlow 1.3 Controller

2. DSCP & TOS Byte Memory Matrix

Severity	DSCP (6-bit)	Bit-Shift	Decimal TOS	Hex TOS	Assigned OVS Queue
CRITICAL	46 (EF)	46 << 2	184	0xB8	Queue 1 (Priority)
ERROR	34 (AF41)	34 << 2	136	0x88	Queue 1 (Priority)
WARNING	26 (AF31)	26 << 2	104	0x68	Queue 0 (Default)
INFO	10 (AF11)	10 << 2	40	0x28	Queue 0 (Default)
DEBUG	0 (CS0/BE)	0 << 2	0	0x00	Queue 0 (Default)

3. Key Commands for Monday Demo Day

```
# 1. Run the interactive terminal showcase (Best for presentation!):
python3 demo.py

# 2. Run the automated 9-test verification suite:
python3 test_suite.py

# 3. Clean up any leftover Mininet virtual networks:
sudo mn -c
```

Good luck with your Monday presentation!

Chronos Log - Computer Networks Mini Project #10 | Built with Python, OpenFlow 1.3 & Ryu